Python: the grammar behind what you've been doing

Syntax and constructs, from values to functions

MSc Digital Scholarship 2026

Two ways to learn a language

Immersion

What we did in Colab: run it, change it, see what happens, read the errors.

```
words = sample.split()
words[-3:]
→ ['knew', 'by', 'noon.']
```

The grammar book

What we do now: name the parts, see the patterns, know what's allowed.

```
thing.method(argument)
list[start:stop]
```

Both help. Each makes sense of the other.

Every value has a type

`str`	string: text	`"Bennet"`
`int`	whole number	`39`
`float`	decimal number	`4.2`
`list`	ordered collection	`['Mr.', 'Bennet']`
`bool`	true or false	`True`

```
type("Bennet")
→ str

type(39)
→ int

"10" + "5"
→ '105'

10 + 5
→ 15
```

= gives a name; == asks a question

= assignment

Make this name refer to this value. Read it as “is now”.

== comparison

Are these two values equal? The answer is True or False.

```
size = len(sample)    # now 196

size == 196           # is it 196?
→ True

name = "Bennet"
name == "bennet"      # capitals!
→ False
```

Three ways to call a tool

FUNCTION

`function(thing)`

Example

`len(sample)`

Read it as

do len to sample

METHOD

`thing.method(…)`

Example

`sample.split()`

Read it as

take sample and split it

MODULE FUNCTION

`module.function(…)`

Example

`statistics.mean(x)`

Read it as

use mean from the statistics module, on x

Compare `sample.split()` with `re.split(pattern, sample)`: similar job, different tool, so the text goes in a different place.

What goes in, what comes back

```
# one argument
len(sample)
→ 196

# two arguments
sample.replace("Bennet", "Darcy")
→ 'Mr. Darcy looked up ...'

# the original is unchanged
sample
→ 'Mr. Bennet looked up ...'
```

Arguments

go inside the brackets, separated by commas.

Return values

are what the tool gives back. Keep one by naming it:

`quiet = sample.lower()`

Showing is not returning

`print(x)` displays x but gives back nothing.

Strings: text and its methods

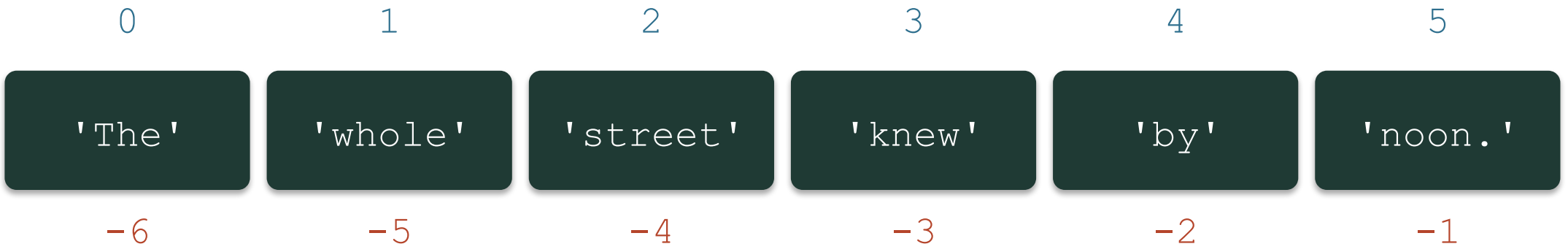
Quotes must match

```
'Mr. Bennet'
"Is that you, my dear?"
'"Is that you?" he asked.'
```

Text containing double quotes goes inside single quotes, and the other way round.

`.lower()`	all lower case
`.upper()`	all capitals
`.count("x")`	how many times x occurs
`.replace("a", "b")`	swap a for b
`.split()`	cut into a list of words
`.strip(".,!?")`	remove characters from the ends
`.startswith("B")`	True or False

Lists: counting from 0, and from the end




```
ending[0]      → 'The'
ending[-1]     → 'noon.'
ending[1:3]    → ['whole', 'street']
ending[-3:]    → ['knew', 'by', 'noon.']
```

`[start:stop]` includes start, stops *before* stop.

Leave a number out to mean from the beginning, or to the end.

`len(ending)` is 6, but the last position is 5.

List comprehensions: make a new list from an old one

```
[len(w) for w in words if len(w) > 5]
```

what to make

len(w): the length of each word

from where

for w in words: take each item in turn, call it w

which ones (optional)

if len(w) > 5: keep only the items that pass

```
[w.upper() for w in words]                  # transform every item
[w for w in words if w.startswith("B")]     # keep some items
→ ['Bennet', 'Bennet']
```

for loops: do something for each item

```
for w in words[0:3]:
    print(w, len(w))
→ Mr. 3
→ Bennet 6
→ looked 6
```

Read it as: *for each of the first three words, call it w, then print w and its length.*

The colon

ends the first line.

The indentation

(four spaces) marks what is repeated. Python uses indentation where other languages use brackets.

Comprehension or loop?

To make a new list: comprehension. To do something, like printing: loop.

Asking questions: True or False

`==`	equal
`!=`	not equal
`<` `>`	less than, greater than
`<=` `>=`	or equal to
`in`	is it inside?
`not in`	is it absent?

```
"Bennet" in words
→ True

if len(w) > 5:
    print("long word:", w)
else:
    print("short word:", w)
```

Writing your own functions

```
def clean(w):
    return
w.strip('.,;:!?"').lower()

clean('"Bennet,')
→ 'bennet'
```

`def` starts a definition, then the name and its inputs

`(w)` the input: a placeholder for whatever is passed in

`:` and an indented body, just like a loop

`return` what the function gives back

Why bother? A function puts a decision, such as what counts as a word, in one named place, where you can read it, test it and report it in your methods.

Modules, packages, libraries

module

one file of Python code you can import: `re`, `statistics`

package

a collection of modules, usually installed first: `nltk`, `pandas`

library

informal: either of the above: "`the pandas library`"

```
import statistics
statistics.mean([3, 6, 6, 2, 4])
→ 4.2

from nltk.tokenize import
sent_tokenize

# in Colab, if a package is
missing:
!pip install something
```

Reading error messages: start at the last line

Error	What it usually means	Typical cause
`NameError`	a name Python doesn't know	`Print(x), sampel, a missing import`
`AttributeError`	no such method for this type	`sample.Upper()`
`IndexError`	position past the end of a list	`words[100]`
`TypeError`	wrong type for this operation	`"Page " + 5`
`SyntaxError`	not valid Python	`a missing quote, bracket or colon`

The worst error has no message: *code that runs and gives the wrong answer.*

Habits for working with AI-written code

1. **Test on something you can check by hand**
   a few sentences where you know the answer
2. **Ask what it assumed**
   what counts as a word, a sentence, the text?
3. **Read the explanation against the code**
   AI explanations can be confidently wrong
4. **Ask twice, ask another AI**
   same prompt, same answer?
5. **Report your decisions**
   they belong in your methods, not hidden in code

Your turn: ask an AI

“Write Python code for Google Colab that downloads Pride and Prejudice from Project Gutenberg and lists the 20 most common words.”

Then: does it run? What did it read? What counts as a word? Did it remove anything? How many times does it count “pride”?